TD/TP13: Correction partie TD

CPES « Sciences des données, arts et cultures »

Antoine GAUQUIER antoine.gauquier@ens.fr
Pierre ABOULKER pierreaboulker@gmail.com

26 mai 2026

1 Exercices théoriques

1.1 Exécution de Dijkstra

Voici le tableau de l'exécution de Dijkstra (Table 1). Chaque ligne correspond à un *pivot*, c'est à dire à un sommet sortant du tas-min Q de l'algorithme (une itération de la boucle While). On indique une croix dans la colonne correspondant au pivot pour se souvenir de ne plus considérer ce sommet dans les prochaines itérations.

On regarde ensuite tous les sommets qui sont dans laliste d'adjacence du pivot : pour tous ceux qui n'ont pas déjà une croix dans leur colonne, on vérifie si la distance du pivot + le poids de l'arête est meilleure (plus petite) que la distance actuelle du sommet considéré. Si oui, alors on met à jour la valeur de distance pour ce sommet dans la ligne du pivot.

Exemple : lorsque D sort de du tas Q à la 6ème itération de la boucle principale While, je mets une crois dans la colonne D de la sixième ligne. Ensuite, je regarde les sommets de la liste d'adjacence de D : $[B, G, H]$. Traitons-les dans cet ordre. Si l'on regarde la colonne de B, il y a déjà une croix dedans : il est donc sorti de Q dans une itération précédente et n'est plus à considérer. Pour G, on a D.dist$= 7$, et $\omega((D, G)) = 7$, donc D.dist$+\omega((D, G)) = 14$. Or, l'actuel G.dist vaut 9 (plus petite valeur de la colonne G), donc ce n'est pas intéressant. Reste alors H : on a H.dist$= +\infty$, donc on fera forcément mieux, et on peut mettre à jour la distance de H avec D.dist$+\omega((D, H)) = 13$.

1.2 Algorithme de Floyd-Warshall

Question 1

Dans Dijkstra, on calcule les plus courts chemins d'une source s à tous les autres sommets (accessibles à partir de s). Pour avoir les plus courts chemins entre toute paire de sommets accessibles, il suffirait d'exécuter Dijkstra sur tous les sommets de V. Cela donne l'Algorithme 1.

Il est facile de déterminer sa complexité : en notant $n = |V|$ et $m = |E|$, on sait d'après le cours que DIJKSTRA est en $O(m \log n)$. On y fait appelle n fois, ce qui donne donc au total une complexité de $O(nm \log n)$.

TABLE 1 – Tableau d'exécution de Dijkstra sur le graphe de la FIGURE 1 du TD13.

pivot	s	A	B	C	D	E	F	G	H
	0	∞	∞	∞	∞	∞	∞	∞	∞
s	X	7	3	4					
B		5	X					11	
C				X	7				
A		X				6			
E						X	8	9	
D					X				13
F							X		
G								X	11
H									X

Algorithme 1 DIJKSTRA sur tous les sommets.

1: **pour** $v \in V$ **faire**
2: DIJKSTRA(G, v)
3: **fin pour**

Attention toutefois : Dijkstra requiert que tous les arcs soient de poids positif ! Or le problème de cet exercice ne fait que l'hypothèse que le graphe G est *sans cycle négatif* : on pourrait avoir des arêtes négatives dans G, et dans ce cas, l'utilisation de l'Algorithme 1 donnerait un résultat incorrect.

Question 2

Il est facile de calculer le nombre de sous-problèmes. La stratégie consiste à calculer le plus court chemin entre chaque paire de noeuds (i, j) en considérant les k premiers sommets (selon un ordre sur V choisi à l'avance, qui peut être arbitraire), en plus des éventuels arcs incluant v_i et v_j. On a donc pour n valeurs possibles pour i, n valeurs possibles pour j, et $n + 1$ valeurs possibles pour k (il est aussi possible de ne considérer que les arcs incluant v_i et v_j, et donc $k = 0$). On a alors $n \times n \times (n + 1) = n^3 + n^2$ sous-problèmes.

Question 3

Les cas de base sont rencontrés quand $k = 0$. Dans cette situation, deux cas de figure : soit $i = j$ et donc on a un unique sommet, soit $i \neq j$ et les deux sommets v_i et v_j sont distincts.

Dans le cas où $i = j$, alors on cherche le plus court chemin d'un sommet à lui-même sans passer par d'autres sommets que $v_i = v_j$. On cherche donc un cycle qui ne passe par aucune arête : la distance est nécessairement nulle. On a alors, dans ce cas, $SP(i, j, 0) = 0$.

Dans le cas où $i \neq j$, on cherche la distance du plus court chemin entre deux noeuds distincts, en ne passant que par des arêtes qui incluent v_i et v_j. Deux sous-cas sont donc possibles :

— $(v_i, v_j) \notin E$. Dans ce cas il est impossible de lier les deux sommets v_i et v_j, puisqu'on ne peut passer que par ces deux sommets et qu'aucune arête n'existe entre eux. On aura alors $SP(i, j, 0) = +\infty$.

— $(v_i, v_j) \in E$. Dans ce cas, le seul chemin existant entre les deux sommets est l'arête (v_i, v_j), et la distance du plus court chemin est alors $SP(i, j, 0) = \omega\left((v_i, v_j)\right)$.

En résumé, on a donc :

$$\forall (i,j) \in |V|^2,\ SP(i,j,0) = \begin{cases} 0 & \text{si } i = j \\ \omega\left((v_i, v_j)\right) & \text{si } i \neq j \text{ et } (v_i, v_j) \in E \\ +\infty & \text{si } i \neq j \text{ et } (v_i, v_j) \notin E \end{cases}$$

Question 4

Comme toujours dans la programmation dynamique, on fait l'hypothèse qu'on a la solution des sous-problèmes à l'ordre inférieur, et on en déduit la solution du problème à l'ordre courant. Dans notre configuration, l'ordre est la valeur de k. On suppose donc que l'on connaît la distance du plus court chemin entre toute paire de sommets i et j en suivant des arêtes incluant les $k-1$ premiers sommets, en plus des sommets v_i et v_j.

On veut alors savoir si inclure le k-ième sommet (noté v_k) permet d'obtenir un chemin plus court que celui trouvé en considérant seulement les $k-1$ premiers sommets, ou si au contraire ajouter v_k ne fait qu'augmenter la longueur des chemins entre i et j. Il existe donc deux cas de figure possibles.

Considérons le cas où l'inclusion de v_k dans le chemin entre les sommets v_i et v_j ne fait qu'augmenter la longueur. Dans ce cas, on a nécessairement $SP(i, j, k) = SP(i, i, k-1)$: en d'autres termes, la longueur du plus court chemin ne change pas, puisque v_k ne peut être intégré sans en augmenter la longueur.

Dans le second cas, on doit intégrer v_k car il permet d'obtenir un chemin dont la longueur est plus petite. On aura alors un chemin de la forme $v_i \to \cdots \to v_k \to \cdots \to v_j$. Ce chemin peut être couper en deux sous-chemins : le sous-chemin $v_i \to \cdots \to v_k$ et le sous-chemin $v_k \to \cdots \to v_j$. Or par définition, chacun de ces sous-chemin ne fait intervenir que $k-1$ sommets : le premier ne peut contenir le sommet v_j et le second ne peut contenir le sommet v_i (si ce n'était pas le cas, alors il existerait un chemin plus court qui ne ferait pas intervenir v_k, et on tomberait dans le premier cas de figure). On connaît donc la longueur de ces deux plus courts chemins, puisqu'ils sont à l'ordre $k-1$. De ce fait, si v_k doit être inclus, alors on a $SP(i, j, k) = SP(i, k, k-1) + SP(k, j, k-1)$.

On a finalement la formule de récurrence suivante :

$$SP(i,j,k) = \min \begin{cases} SP(i,j,k-1) & \text{si } v_k \text{ n'est pas inclus dans le plus court chemin} \\ SP(i,k,k-1) + SP(k,j,k-1) & \text{si } v_k \text{ est inclus dans le plus court chemin} \end{cases}$$

, dans la mesure où on choisit le cas de figure qui minimise la longueur du chemin.

Question 5

L'Algorithme 2 présente le pseudo-code correspondant au problème de programmation dynamique présenté jusque là.

Algorithme 2 FLOYD-WARSHALL

Entrée: Un graphe $G = (V, E, \omega)$ avec $V = \{v_1, \ldots, v_n\}$
Sortie: La matrice des plus courts chemins D

```
1:  Initialiser D tableau de taille n × n × (n + 1)
2:  pour i ← 1 à n faire
3:      pour j ← 1 à n faire
4:          si i = j alors
5:              D[i][j][0] ← 0
6:          sinon
7:              si (v_i, v_j) ∈ E alors
8:                  D[i][j][0] ← ω((v_i, v_j))
9:              sinon
10:                 D[i][j][0] ← +∞
11:             fin si
12:         fin si
13:     fin pour
14: fin pour
15: pour k ← 1 à n faire
16:     pour i ← 1 à n faire
17:         pour j ← 1 à n faire
18:             D[i][j][k] ← min(D[i][j][k − 1], D[i][k][k − 1] + D[k][j][k − 1])
19:         fin pour
20:     fin pour
21: fin pour
        retourner D
```

Question 6

La complexité spatiale est en $O\left(n^3\right)$, correspondant au tableau D. La complexité temporelle est en $O\left(n^3\right)$, avec les trois boucles for imbriquées aux lignes 15–21. La double boucle imbriquée des lignes 2–14 est en $O\left(n^2\right)$. L'initialisation de D est (au pire cas, dépendemment de comment cela est fait) en $O\left(n^3\right)$. Le total est donc en $O\left(n^3\right)$.

Question 7

Premièrement, cette solution est de toutes façons préférable si au moins un arc de G est de poids négatif, car DIJKSTRA retourne des solutions incorrectes dans cette situation.

FLOYD-WARSHALL est également intéressant en termes de complexité temporelle si le graphe G est *dense*, ou proche de l'être. Un graphe est *dense* si $m \approx n^2$, c'est à dire que chaque sommet est connecté à une grande majorité d'autres sommets dans le graphe.

Dans un tel cas, alors la complexité obtenue en **Question 1** devient :

$$O\left(nm\log n\right) \approx O\left(n \times n^2 \times \log n\right) \approx O\left(n^3 \log n\right) > O\left(n^3\right)$$